OSU Satellite TV for MITRE eCTF 2025

Contents

1. Team

The team is comprised of the following people from the Ohio State State University:

Student	Advisor
Christopher Begines	Julia Armstrong
Mark Bundschuh	Dr. Felix Engelmann
William Comer	
Mason Erwine	
Aditya Gupta	
Ryan Jackman	
Cadan Keller	
Jack Leverett	
Christopher McDevitt	
Maxwell McGee	
Jacob Mcquade	
Diego Noria	
Divij Sasidhar	
Joshua Sims	
Eva Smith	
JT Vendetti	
Matthew Weikel	
Zihao Zhang	

2. Introduction

This document describes OSU's implementation of a secure satellite TV system as specified by MITRE's eCTF 2025. The Satellite TV system features a simulated satellite sending ASCII art to subscribed hardware Decoder devices, implementing the Functional Requirements (Section 3). The decoder devices are MAX78000FTHR microcontrollers by Analog Devices. The system is secured according to the Security Requirements (Section 4), and designed to defend against attacks as outlined in Attack Scenarios (Section 5), which revolve around erroneously decoding frames sent by a satellite.

2.1. Source Code and Documentation

Source Code: https://github.com/OSU-embedded-security-club/ectf-osu-2025
Documentation: https://osu-embedded-security-club.github.io/ectf-osu-2025

3. Functional Requirements

The two main components are the Encoder and Decoder. The encoder receives frames of up to 64 bytes, and converts them into our custom format before sending it over the satellite link. Then, each decoder receives the frame, and must decode the message into the original 64 bytes.

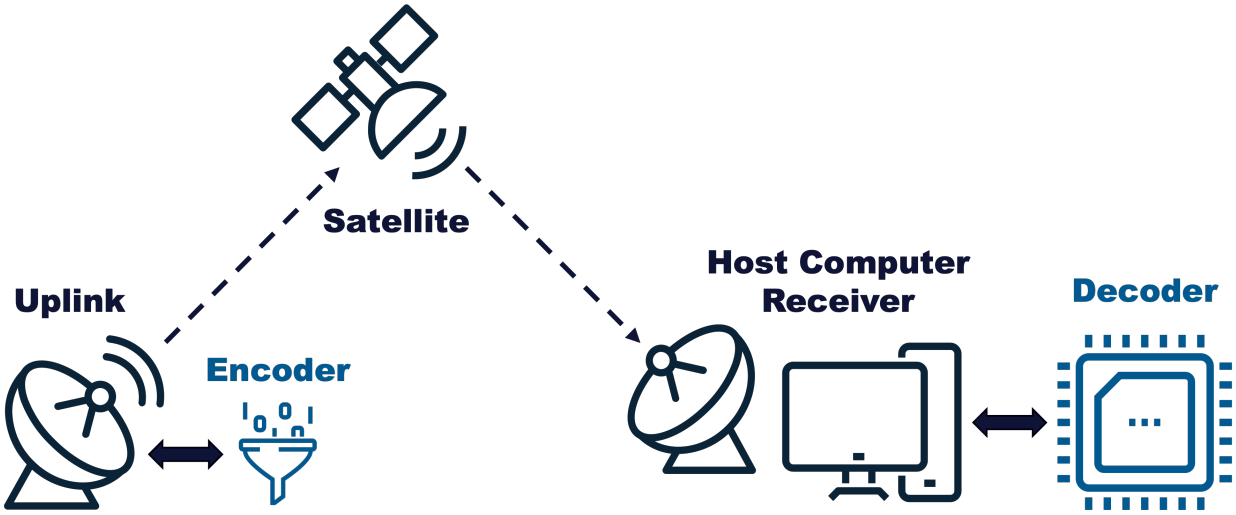

Figure 1: Full system with blue highlights indicating components which need to be implemented. [1]

3.1. Functional Requirement 1: List Channels

"The Decoder must be able to list the channel numbers that the Decoder has a valid subscription for." [1]

3.2. Functional Requirement 2: Update Subscription

"The Decoder must be able to update it's channel subscriptions with a valid update package. If a subscription update for a channel is received for a channel that already has a subscription, the Decoder must only use the latest subscription update." [1]

3.3. Functional Requirement 3: Decode Frame

"The Decoder must properly decode TV frame data from any channel that has a valid and active subscription installed or for any emergency broadcast frames." [1]

3.4. Available Channels

A decoder must be able to store up to 8 subscriptions, which must persist over reboots. Additionally, Channel 0 is reserved as an Emergency Channel which all decoders must be able to decode, as if they always had an active subscription. The deployment may be provisioned with up to $2^{32} - 1$ channels.

4. Security Requirements

4.1. Security Requirement 1: Active Subscription

"An attacker should not be able to decode TV frames without a Decoder that has a valid, active subscription to that channel." [1]

See Section 7.1 on how the design protects against the security requirement.

4.2. Security Requirement 2: Authentication

"The Decoder should only decode valid TV frames generated by the Satellite System the Decoder was provisioned for." [1]

See Section 7.3 on how the design protects against the security requirement.

4.3. Security Requirement 3: Increasing Timestamps

"The Decoder should only decode frames with strictly monotonically increasing timestamps." [1]

See Section 7.4 on how the design protects against the security requirement.

5. Attack Scenarios

We consider 5 different scenarios where the attackers have access to certain information that the system is designed to protect against.

5.1. Attack Scenario 1: Expired Subscription

An attacker has physical access to a decoder with a valid subscription to a channel. The timestamp of the current frames sent by the encoder is past the ending timestamp of the attacker's subscription.

See Section 7.1 on how the design protects against the attack scenario.

5.2. Attack Scenario 2: Pirated Subscription

An attacker has physical access to a decoder, and a valid subscription to a channel for a different decoder which they do not have physical access to.

See Section 7.2 on how the design protects against the attack scenario.

5.3. Attack Scenario 3: No Subscription

An attacker has physical access to a decoder, but has no subscriptions.

See Section 7.1 on how the design protects against the attack scenario.

5.4. Attack Scenario 4: Recording Playback

An attacker has physical access to a decoder, and has been recording the raw data sent by the satellite for some time. Then, they get a valid subscription for the channel. In other words, the attacker should not be able to read past frames given an active subscription.

See Section 7.1 on how the design protects against the attack scenario.

5.5. Attack Scenario 5: Pesky Neighbor

An attacker has physical access to a decoder, and spoofs the satellite signal to a neighbor's decoder which has a valid subscription. The attacker does not have physical access to the neighbor's decoder.

See Section 7.3 on how the design protects against the attack scenario.

6. Deployment

See the `README.md` at the root of the repository for instructions on setting up dependencies and using the `Taskfile` to execute deployment commands.

6.1. Secrets

The following secrets are generated for a deployment. Once generated, they are global and read only. The deployment secrets are available to the encoder. Some of these secrets will be shared with the Decoders in various forms as described by Section 6.2.

- $S_1, ..., S_8$
  - Up to 8 32 byte randomly generated seeds, one per encrypted channel

- $S_{\text{subscription}}$
  - Randomly generated 64 byte subscription salt
- $K_{\text{secret}}, K_{\text{public}}$
  - An Ed25519 asymmetric key pair
- K_{metadata}
  - Randomly generated 32 byte shared symmetric key

6.2. Decoder

The firmware must be built for each separate decoder device. When building, specify the `DEVICE_ID` environment variable as a 4 byte hexadecimal number starting with `0x`, denoted as D_{id}.

At build time, the decoder derives the following secrets and stores them within the encrypted firmware:

- K_{metadata}
  - Shared 32 byte symmetric key with the Encoder which is used to encrypt the metadata (timestamp, channel id) for decode packets
- K_{public}
  - Shared public key with the Encoder
- $K_{\text{subscription}}$
  - Derived as $\text{blake3}\left(S_{\text{subscription}} + D_{\text{id}}\right)$, and is the symmetric key used to encrypt and decrypt subscriptions for a particular decoder
- K_{flash}
  - Randomly generated 32 byte symmetric used to encrypt data before writing it to the flash storage

7. System Design

7.1. Encode and Decode

Field Name	Size	Relative Size
Signature	64 bytes	
Channel Id	4 bytes	
Timestamp	8 bytes	
Message	up to 64 bytes	

Table 1: Structure of a decode packet

Each frame has an integer timestamp $t \in [0, 2^{64} - 1]$. We derive a unique key k_t for each timestamp which the encoder will use to encrypt the *Message*, and the decoder will use to decrypt the *Message*, both using Salsa20. The keys are derived with a *binary hash key derivation tree*. This ia a binary tree with the channel's seed value, $S_{\text{channel\_id}}$ at the root. For each node, the left child is $L(x) = \text{blake3}(x + \text{“L”})$ and the right child is $R(x) = \text{blake3}(x + \text{“R”})$. Figure 2 is a diagram of a small hash key derivation tree for $t \in [0, 7]$. For example, the key for timestamp 3 is computed using 3 hashes, $k_3 = h_{011} = R(R(L(S_{\text{channel\_id}})))$. Note that the full sized tree has a height of 64 and 2^{64} leaves, so the encoder can calculate the frame's key with 64 hashes. The encoder encrypts *Message* with the frame's key with Salsa to form *EncryptedMessage*. Since each key is only used once, this encryption always uses 0 as a nonce.

For channel 0, there are no subscriptions, so this first layer of encryption doesn't occur, and *EncryptedMessage* = *Message*.

After the *Message* is encrypted with frame's key, *Channel Id*, *Timestamp*, and *EncryptedMessage* are signed with K_{private} to form the *Signature* which is prepended to the data. Then, those three fields are Salsa encrypted with the symmetric K_{metadata} key using the first 8 bytes of *Signature* as a nonce. The signature is concatenated with the encrypted *Channel Id*, *Timestamp*, and *EncryptedMessage* to form the final message.

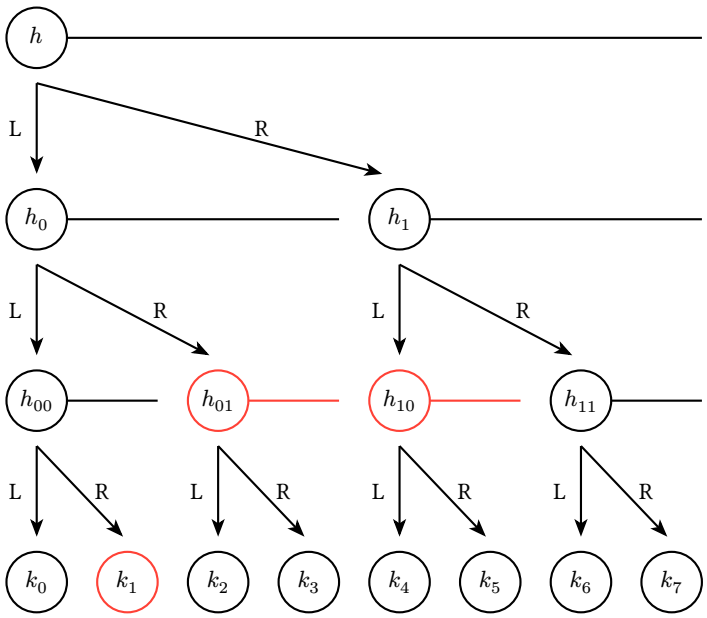

Figure 2: A hash key derivation tree from $t = 0$ to $t = 7$. The 4 red nodes would be sent to the decoder for a subscription from $t = 1$ to $t = 5$.

To allow the decoder to decrypt frames in its subscription, subscription packets contain nodes from the tree that cover the subscription's range, but not any timestamps outside of the range. These nodes are called "roots". For example, the nodes highlighted in red in Figure 2 are the roots for a subscription from $t = 1$ to $t = 5$. $h_{001} \Rightarrow \{k_1\}$, $h_{01} \Rightarrow \{k_2, k_3\}$, and $h_{10} \Rightarrow \{k_4, k_5\}$, so only h_{001}, h_{01}, and h_{10} are needed to form the subscription. To decode a frame inside the subscription, the decoder computes only the hashes required to go from a root to a leaf node, then uses that result to decrypt the frame data.

This aspect of the design secures the system in the following ways:

- **SR1**: Active Subscription
  - The decoder is never given more keys than exactly those which correspond to the timestamps in the subscription it was given. This means it is cryptographically impossible to decrypt frames outside of the subscription interval.
- **AS1**: Expired Subscription
  - The decoder is never given keys for the frames past the end of the subscription interval, so it is cryptographically impossible to decrypt frame data past the end of its subscription.
- **AS3**: No Subscription
  - Attackers without any subscription have no keys for any frames, so they are unable to decode any frames at all.

- **AS4**: Recording Playback
  - The decoder is never given keys for the frames outside of the subscription interval, so it is not possible to decrypt past frames.

7.2. Subscription

Field Name	Size	Relative Size
Signature	64 bytes	
Start Timestamp	8 bytes	
End Timestamp	8 bytes	
Channel Id	4 bytes	
Root Hashes	$\{24n \text{ bytes} \mid n \leq 126\}$	 $\ldots n$ times

Table 2: Structure of a subscribe packet

To generate a subscription, the encoder calculates the root hashes to send to the decoder that will allow it to decode only frames in the timestamp. The locations of the roots are computed using the algorithm described in Section 8.2. A root location is defined by its offset (starting timestamp) and its power (how high it is in the tree). Its length is 2^{power}. Then the encoder calculates the hashes for each root, starting at the root node and going down to the root's power, doing left or right hashes based on the bits of the roots's offset.

The plaintext contents *Start Timestamp*, *End Timestamp*, *Channel Id*, and *Root Hashes* are signed with K_{private} to form the *Signature* which is prepended to the data. Then, those four fields are encrypted with $K_{\text{subscription}}$ using the first 8 bytes of *Signature* as a nonce, which forms the final subscribe packet. This ensures that subscriptions can only be used by the decoder for which the subscription was generated.

The decoder runs the same algorithm as the encoder to locate the root nodes from the start and end timestamps, so the subscription packet can contain only the hash values, with no extra metadata like root locations or number of roots. The proof in Section 8.1 shows that this scheme requires no more than 126 root nodes. See Section 8.2 for the algorithm which is used to find the root nodes corresponding to a timestamp range.

Unlike in Section 7.1, the *Channel Id* may not be 0, because every decoder is assumed to already be subscribed to Channel 0 (the Emergency Channel).

This aspect of the design secures the system in the following ways:

- **AS2**: Pirated Subscription
  - Encrypting the subscription file with $K_{\text{subscription}}$ means that an attacker needs physical access to the decoder which a subscription was generated for. Additionally, signing the subscription with asymmetric cryptography ensures that further subscriptions are created by the encoder and cannot be forged by other decoders. In the Pirated Subscription scenario, the attacker is given a subscription file for a device id which they do not have, so it is not possible to extract the key and decrypt the subscription to apply it to the attacker's own decoder.

7.3. Signatures

The encoder uses the secret key SK to sign every message with a 64 byte Ed25519 signature. The public key PK is stored within all decoders as a static variable in the `.rodata` section and verifies every message to ensure it came from the encoder it was provisioned with.

This aspect of the design secures the system in the following ways:

- **SR2**: Authentication
  - Using asymmetric cryptographic, the decoder can ensure that it only accepts messages from the encoder it was provisioned with. Additionally, it prevents a compromised decoder from sending messages to other decoders, unlike symmetric cryptography.
- **AS5**: Pesky Neighbor
  - Bypassing the signature would require an attacker to write their own public key into the decoder which they want to receive messages on. In the Pesky Neighbor attack scenario, attackers do not have physical access to the neighbor's decoder, meaning that no side channel or other embedded attacks are possible. Additionally, using Zig, we can have memory safety guarantees which prevent memory vulnerabilities required to write to overwrite the neighbor's public key.

7.4. Global Timestamp

A global timestamp is kept in the RAM of the decoder, and is used to reject misordered frames. It increases on every successful decode.

This aspect of the design secures the system in the following ways:

- **SR3**: Increasing Timestamps
  - The timestamp can be checked against the timestamp in the frame before attempting to decode the data in frame, ensuring that the decoder never decodes an earlier frame.

7.5. Memory Protection Unit

The Memory Protection Unit (MPU) is enabled on the decoder as a defensive countermeasure to code injection and serves as an additional layer of security. We disable writing to flash, and prevent executing in SRAM.

7.6. Encryption at Rest

Before writing to flash, the contents of the page are first encrypted with a symmetric key K_{flash} using Salsa20. K_{flash} created by the decoder at build time and stored within the encrypted firmware (see Section 6.2). An incrementing nonce is stored next to each page in plaintext, to prevent nonce reuse. This adds another layer of security aiming to prevent offline attacks where a sophisticated attacker is able to dump the flash of the decoder and brute force the subscription's root hashes.

7.7. Zig

The Zig programming language was selected as the language to write the secure decoder in. Zig has memory safety features like protection from indexing an array out of bounds, double free protections and more. Additionally, it includes a comprehensive standard library which securely implements many cryptographic functions, among many other quality of life functions. Most importantly, Zig can do all this while maintaining perfect C compatibility and using the MSDK provided by Analog Devices Inc (the developer of the MAX78000FTHR), without requiring a rewrite of the Hardware Abstraction Layer (HAL). This allowed the team to move incredibly quickly while ensuring a high level of security.

8. Appendix

8.1. Proof that any interval can be encoded with at most 126 roots

By the Theorem (Section 8.1.4) proved below, any interval $[a, b]$ can be encoded by at most $2n - 2$ roots. Timestamps are 64 bit unsigned integers, so we choose $n = 64$. Therefore, no more than $2(64) - 2 = 126$ roots are required to encode any interval on $[0, 2^{64} - 1]$.

8.1.1. Definitions

We will start by making a few mathematical definitions for the concepts of roots and trees.

A *root*, which is the location of a node in the hash key derivation tree, is a tuple (o, l) where l is a power of 2 and o is a multiple of l. Its *power* is $\log_2(l)$. (If o is 0, l can be any positive power of 2).

A root (o, l) *contains* a timestamp x if $o \leq x \leq o + l - 1$.

A *tree* is a finite set of roots (these would sent in a subscription packet to the decoder).

To say that an interval I can be *encoded by* a tree T is to say that $\bigcup_{(o,l)\in T}[o, o + l - 1] = I$.

We will prove that any interval in a tree of height n can be encoded by a tree with at mosts $2n - 2$ roots.

8.1.2. Lemma 1

For all $n \geq 1$, any interval $[0, k]$, where $k \leq 2^n - 1$, can be encoded by a tree containing at most n roots.

Base case, $n = 1$:

We want to show that any interval $[0, k]$ where $k \leq 2^1 - 1 = 1$ can be encoded by at most 1 root.

- $\{(0, 1)\}$ encodes $[0, 0]$
- $\{(0, 2)\}$ encodes $[0, 1]$

Induction hypothesis: Suppose any interval $[0, k]$ where $k \leq 2^n - 1$ can be encoded by a tree with at most n roots.

We wish to show that any interval on $[0, b]$ where $b \leq 2^{n+1} - 1$ can be encoded by a tree with at most $n + 1$ roots.

Let $I = [0, b]$ where $b \leq 2^{n+1} - 1$.

Case 1: $b < 2^n$: By the induction hypothesis, I can be encoded with a tree with at most $n \leq n + 1$ roots.

Case 2: $2^n \leq b < 2^{n+1}$: Then $I = [0, 2^n - 1] \cup [2^n, b]$.

The first interval, $[0, 2^n - 1]$ can be encoded by 1 root, namely, $(0, 2^n)$.

Since $b - 2^n < 2^n$, we can shift the second interval $[2^n, b]$ to the left by 2^n, obtaining the interval $[0, b - 2^n]$, which by the induction hypothesis can be encoded by a tree T containing at most n roots.

Shifting that tree back to the right, we obtain $T' = \{(o + 2^n, l) \mid (o, l) \in T\}$ which encodes $[2^n, b]$. T' has the same number of roots as T, so T' has at most n roots.

Therefore, I is encoded by $\{(0, 2^n)\} \cup T'$, which contains at most $n + 1$ roots.

8.1.3. Lemma 2

For all $n \geq 1$, any interval $[k, 2^n - 1]$ can be encoded with at most n roots.

Let T be the tree which by Lemma 1 encodes $[0, 2^n - 1 - k]$ and contains at most n roots. Then let $T' = \{(2^n - l - o, l) \mid (o, l) \in T\}$, which also contains at most n roots. Intuitively, this is the tree T flipped horizontally.

We will show that T' encodes $[k, 2^n - 1]$ by considering an arbitrary timestamp $x \in [k, 2^n - 1]$. $2^n - 1 - x$ is within the interval $[0, k]$, so there exists a root $(o, l) \in T$ which contains $2^n - 1 - x$, that is, $o \leq 2^n - 1 - x \leq o + l$. Therefore, $2^n - 1 - l - o \leq x \leq (2^n - 1 - l - o) + l$, so the corresponding root in T', $(2^n - l - o, l)$, contains x.

8.1.4. Theorem

For all $n \geq 2$, any interval on $[0, 2^n - 1]$ can be encoded by at most $2n - 2$ roots.

Base case, $n = 2$:

We want to show that all intervals on $[0, 3]$ can be encoded by at most 2 roots.

- $\{(x, 1)\}$ encodes $[x, x]$ for $x \in [0, 3]$
- $\{(0, 2)\}$ encodes $[0, 1]$
- $\{(0, 2), (2, 1)\}$ encodes $[0, 2]$
- $\{(0, 4)\}$ encodes $[0, 3]$
- $\{(1, 1), (2, 1)\}$ encodes $[1, 2]$
- $\{(1, 1), (2, 2)\}$ encodes $[1, 3]$
- $\{(2, 1), (3, 1)\}$ encodes $[2, 3]$

Induction hypothesis: Suppose any interval on $[0, 2^n - 1]$ can be encoded by at most $2n - 2$ roots.

We wish to show that any interval on $[0, 2^{n+1} - 1]$ can be encoded by at most $2(n + 1) - 2 = 2n$ roots.

Let $I = [a, b]$ be a interval on $[0, 2^{n+1} - 1]$.

Case 1: $a \leq b < 2^n$. Intuitively, this means the range is within the left half of the tree. By the induction hypothesis, I can be encoded by $2n - 2 \leq 2n$ roots.

Case 2: $2^n \leq a \leq b$. Intuitively, this means the range is within the right half of the tree. The interval $[a - 2^n, b - 2^n]$ is a subinterval of $[0, 2^n - 1]$, so by the induction hypothesis, there is a tree T containing at most $2n - 2$ roots which encodes $[a - 2^n, b - 2^n]$. Thus, the tree $T' = \{(o + 2^n, l) \mid (o, l) \in T\}$, also containing at most $2n - 2$ roots, encodes $[a, b]$.

Case 3: $a < 2^n \leq b$. Intuitively, this means the interval crosses the center of the tree. So $I = [a, 2^n - 1] \cup [2^n, b]$. By Lemma 2, $[a, 2^n - 1]$ can be encoded by a tree T containing most n roots. Similarly, by Lemma 1, $[0, b - 2^n]$ can be encoded by can be encoded by a tree S containing at most n roots, so $S' = \{(o + 2^n, l) \mid (o, l) \in S\}$ encodes $[2^n, b]$. Therefore, I can be encoded by $T \cup S'$, which contains at most $n + n = 2n$ roots.

8.2. Algorithm to calculate root node positions in the hash key derivation tree

This algorithm computes the positions of the smallest set of nodes in the hash tree that allows the computation of all keys within the range of timestamps $[a, b]$ (inclusive), but none outside of that range. The same algorithm is implemented in the subscription generator in Python and on the decoder in Zig. It works by repeatedly adding the highest possible root that doesn't go too long while iterating over the range from left to right. The length of a node must be a power of 2 multiple of its offset, which is why the power is limited by the number of trailing zeroes of the offset. We also make sure that the power does not go to long, by ensuring that the starting location plus the length is not larger than the ending timestamp.

```python
class Root:
    """
    The position of a node in the hash key derivation tree.

    Offset is the starting timestamp of the node.
    Power is its position vertically. A power of 0 means a node at the bottom of a
    tree (representing an individual timestamp's key), and a power of 64 is at
    the top (representing the entire range of timestamps).

    A node's offset is always a multiple of 2**power.

    Knowing the value of a hash at a root allows one to compute timestamps
    in the range [offset, offset+2**power - 1] (inclusive).
    """
    offset: int
    power: int


def get_roots(a: int, b: int) -> list[Root]:
    if a > b:
        raise ValueError("Invalid range: a > b")

    ranges = []

    while a <= b:
        # `ctz(a)` (count trailing zeroes) is the largest
        # that the power can be due to the alignment of `a`.
        # Also, we need `a + 2**power - 1 <= b` so the range doesn't go too long,
        # which implies `power <= (b - a + 1).bit_length() - 1`.
        power = min((b - a + 1).bit_length() - 1, ctz(a))
        ranges.append(Root(power=power, offset=a))
        a += 2 ** power

    return ranges
```

Bibliography

[1] MITRE, "2025 eCTF Documentation." [Online]. Available: https://rules.ectf.mitre.org/2025